

The Extension of User Story Template Structure with an Assessment Question Based on the Kano Model

Grażyna Hołodnik-Janczura

Abstract The aim of this study was to verify whether a simple method of customer satisfaction survey can aid Agile project management in an effort to provide customers with an IT solution that is well-matched to their actual needs. A pilot study was made to confirm the frequent occurrence of the problem under consideration. Based on literature review, an analysis of the relationship between user requirements and final properties of the software was made, starting from user goals, and ending with ways to ensure user satisfaction. Discussed was also the impact of customer business value on customer satisfaction from the use of the resulting IT solution. The described importance of the business value resulted in pointing to the need for combining Agile and Lean principles, leading to effective elimination of waste. Product backlog was characterized as one of the key artifacts to the Agile approach. This backlog became the foundation for the draft of a new user story template structure, as well as for its application in waste elimination as soon as at the stage of formulating the needs and requirements of the customer/user. The end of the study presents an example usage of a template designed in this manner.

Keywords Customer satisfaction • Elimination • Feature • Product backlog • Template • User story • Waste

1 Introduction

Taking up the issue of elimination of unnecessary features of computer application was inspired by author's own observations and other published data. Users of various computer applications often complain that software contains many features that they do not use. It was even found that only about 20 % functions on typical client software are frequently used, and 2–3 % are rare features. Thereby, the utility index of such products reduces [1, p. 24]. What occurs here is a waste of effort in

G. Hołodnik-Janczura (✉)

Wrocław University of Science and Technology, Wrocław, Poland
e-mail: Grazyna.Holodnik-Janczura@pwr.edu.pl

successive phases of a complex process, including coding, testing, documenting and, consequently, working code maintenance. Therefore, studying the adequacy of product features to the real needs of the customer plays an increasingly important role. To meet such expectations, a pilot study to confirm or reject this opinion was designed and conducted.

Additionally, knowledge of the application of the Kano model for customer satisfaction analysis in Agile projects reinforced with Lean principles was the source of the concept behind the attempt at solving this issue. The solution is designed as a combination of the two sides of the problem under examination, that is, the formulation of customer/user needs and requirements in the form of user story with their simultaneous value assessment from the point of view of the customer/user.

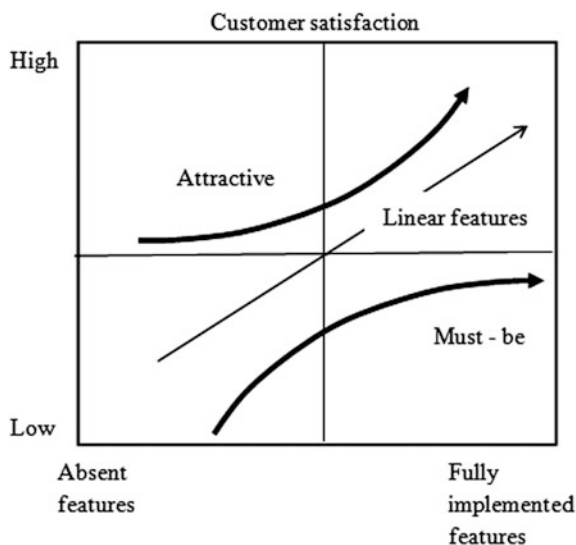
2 The Relationship Between Customer Satisfaction and the Degree to Which Customer Needs Are Met

Modern approach to ensuring customer satisfaction puts customer goals—that the customer intends to achieve by purchasing or ordering a new product—at the top of the pyramid of knowledge about customer needs. The methods of customer satisfaction study in the Agile approach are often based on the Kano model (Fig. 1) or MoSCoW method, both based on prioritization. Other methods, such as the very detailed and time-consuming QFD method, are not recommended by the generally agile and economical Agile approach. Noriaki Kano, consultant of the Japanese TQM, developed a customer satisfaction model in relation to the product features, distinguishing three categories of features based on customer preference assessment [2, p. 79]:

- must be: “expected” features, the presence of which the customer takes for granted, their absence is inevitably followed by customer dissatisfaction; e.g. inoperative part of the software, missing instructions. These features are alleged, the customer does not feel the need to express them, they are “obvious” and their absence causes irritation;
- linear: the features that the customer asks for and tells about; the more these features in a product, the greater customer satisfaction; e.g. greater efficiency, lower cost, higher reliability, easier to use;
- attractive: these product features give the customer a pleasant surprise, yet if they are absent, the client will not be dissatisfied; the customer does not express the desire to have such extras, or may not know how to express such needs, e.g. because of a lack of technical knowledge.

Customer satisfaction is inextricably linked with the concept of product quality. Product quality equals the degree to which customer needs are met; it is, namely, the core element of a universal product quality definition: “Quality is the degree to which a set of inherent features fulfills the requirements” [3]. Comparing the

Fig. 1 Customer satisfaction model according to Kano.
 Source Author, based on [8, p. 113]



classification of Kano against this ISO definition, it is necessary to elaborate on the meaning of an inherent feature. It is a permanent feature of the product, process or system. Sample inherent features of a car are: amount of fuel consumed, maximum engine power. The term “requirement” means a need or expectation that is: stated, mandatory or customary.

Stated requirements are formulated by the customer in various ways, e.g. through negotiations with the supplier or as a result of marketing research. Sample requirements fixed by the customer in a car showroom are: the brand and model of car, number of doors, and engine power.

Mandatory requirements are such requirements that the supplier must meet regardless of whether the customer formulates them or not. They result from legal provisions, e.g. in the field of safety, health protection, fire protection. For example, the car must be made in accordance with applicable safety regulations or traffic admission regulations.

Customary requirements usually are not formulated by the customer, because of a general assumptions that the provider should fulfill them due to the current level of technical culture, and that they will be obviously met. However, an example of such a requirement may for one customer group be a built-in car radio, while another group may find such a feature excessive. This is a group of features subjectively classified as common, depending on the habitual use of facilities.

Thus, in the context of the Kano model, the mandatory requirements are perceived as “must be” and do not require further study. In contrast, the stated requirements can be assessed differently by various customers, likewise the customary requirements. Some customers will recognize such features as unexpected attractions, while other will see them as universal, which means they should be subject to further studies in order to be qualified as waste and eliminated.

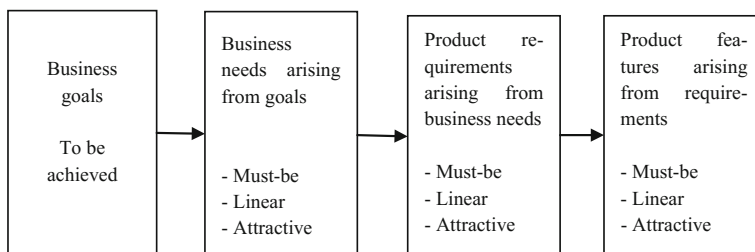


Fig. 2 Relationship between business goals and product features expressed through needs and requirements and their classification. *Source* Author, based on [4]

Due to the fact that the above definition of ISO is of universal character, quality models specific to IT solutions should be mentioned here. There are quite many, including: FURPS (+), IEEE standards, ISO/IEC 9126, ISO/IEC 15504. These models and standards are rooted in the original two: McCalla model (1977) and Boehm model (1978). In each of them, quality evaluation of an IT solution depends, similar to ISO standards, on the fulfillment of customer/user needs and requirements, and is measured by means of a corresponding set of features. In order to emphasize the existing relation between requirements quality and the quality of the final product, an organized group of IT quality standards known as SQaRE [4] should be brought up. The SQaRE model represents these significant relations (Fig. 2).

The SQaRE model supports the occurrence of full dependence between the goals and business needs of project stakeholders, business needs and requirements, and consequently, the requirements and features of the product. The second conclusion is that the classification of these elements follows the same lines. Such an approach to customer satisfaction is considered the basis for deliberations on providing customers and software users only such properties that meet their exact needs. In contrast, providing a product with excess or inadequate features with respect to the actual needs is considered waste, causing higher costs and longer implementation deadlines.

3 The Use of Application Features Provided for the User: Study Results

The questionnaire study focused on the suitability of features of computer applications in use by the respondents. The questionnaire was posted on the website¹ of the author of this study, who received 12 filled questionnaires. Respondents had different, though generally long (between 9 and 36 years) experience of work with

¹www.ioz.pwr.wroc.pl/Pracownicy/holodnik.

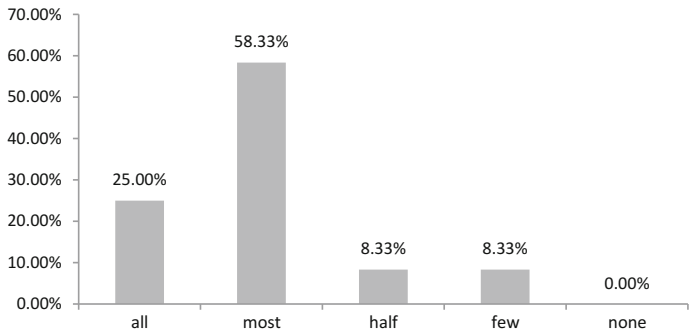


Fig. 3 Descriptive assessment of all used applications based on the content of unnecessary features. *Source* Author

the use of computer software for purposes such as work, education, entertainment, and handling of official matters. All respondents use between 15 and 100 various applications daily. The assessment of the satisfaction level on a 6-point scale, from very high level to total lack of satisfaction with the software used, was quite varied and depended on the type of software. Majority of responses pointed to high (9 respondents) and medium (7 respondents) levels of satisfaction, while the minority of responses pointed to the lowest level, or lack of satisfaction (2 respondents). All respondents claimed that significant parts of the software were unnecessary, that is, unused—e.g. 25 % of respondents think that all applications contain unnecessary features, and over 58 % think that most applications contain unnecessary features (Fig. 3).

When asked about how many of the features of used applications were unnecessary, 23 % of respondents said it was 60, 30.77 % of respondents said 30 %, and only 7.69 % of respondents think 10 % were unnecessary features (Fig. 4).

This short pilot study, although small, but conducted on a representative group of software users, confirmed other published and observed claims that the software market is filled with applications that have a significant number of unnecessary features.

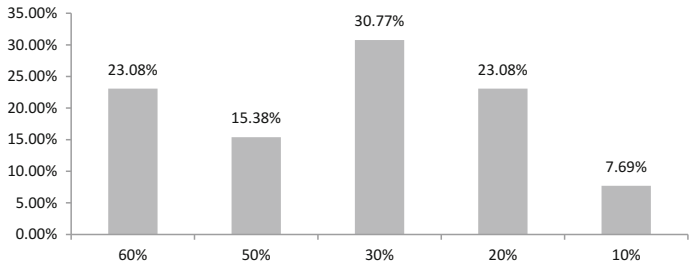


Fig. 4 Application features considered unnecessary in the software used by respondents. *Source* Author

4 Business Value as a Way to Increase Customer Satisfaction

Managing customer value is one of the marketing concepts from the wide area of management processes. In the 80s, Jack Welsh popularized the idea of value management [5] as one way to achieve customer satisfaction. In subsequent years, this concept has been well developed and increasingly used. The tools used to review value are primarily quantitative, but also semi-quantitative methods of financial analysis, since value can also be perceived subjectively, allowing a subjective assessment of product features by specifying: this is something I like, dislike, or feel neutral about.

In the management of IT projects, which are also subject to investment efficiency, the concept works in a similar way. Customer value, in IT projects referred to as business value, reflects the financial benefits expected from the implementation of a computer application, and resulting from growth in revenue or cut in spending. Financial indicators—such as ROI, NPV, IRR—used to determine the cost-effectiveness of alternative project drafts allow to take into account the concept of business value also in terms of IT project efficiency. On the client side, business value of a computer application is difficult to estimate, because it is not seen directly—contrary to income from sales. Business value is reflected in how it impacts the outcome of company operations, and is perceived as indirect and supporting the sales. Estimation of this impact is only possible with respect to a certain period of time, i.e. months or years.

On the other hand, the development of an IT project should also be subordinate to its needs and taking into account the perspective of business value. Hence, the question arises about the value of the features of the expected software. In Agile projects, when the software is delivered in iterative-incremental cycles, it is possible to decide on the order of delivered features, depending on their value to the customer. While the business value of a software investment calculated as a whole can be conducted with the use of methods accounting investment profitability, splitting this value between each release of the software in the following increments is a separate issue. The problem lies in the search for relationships between different parts, and in delivering them to the expected customer value and correct sequence from the viability point of view. The issue of relationship between releases is solved through the use of prioritizing. The priority is determined in direct relation to customer value.

In Agile projects, where elimination of all kinds of waste is necessary, the priority value should be determined quickly and cheaply, without involving complex and expensive methods. Commonly used are simple and often subjective methods, which is not an obstacle, but a benefit, as it is the customer who determines the final priority value, best from his point of view.

5 Strengthening Value Managements by Connecting Agile and Lean Approaches

There is a natural relationship between value and need: if something is needed, it is because it has value. Therefore, intending to introduce the concept of eliminating unnecessary software features, the basic principles of Agile and Lean approaches will be briefly discussed. These approaches have one common goal: to increase customer satisfaction. The guiding principle of the Agile Manifesto: “Our highest priority is to satisfy the customer through early and continuous delivery of valuable software” clearly and explicitly indicates the key element of this approach [6].

The concept of value has been highlighted in the Lean Thinking approach, which adopted the following basic principles in production management: Value, Value Stream, Flow, Pull and Perfection [7]. The notion of value was clarified by the introduction of its opposite—waste; hence, “everything that is not value is waste” [7, p. 14]; the principle of waste elimination was accepted as the basis of all activities leading to improved customer satisfaction. Lean Approach was adopted to project management in the wake of production systems, since software development processes evolve similarly. Changes made since the second half of the 80’s have resulted in the establishing of first the Agile approach, whose basis² is consistent with Lean ideas to a degree that led to creating the concept of Lean Software Development.

For an Agile project to be considered Lean, it is necessary to adopt the five Lean principles into project management [7, p. 13]:

1. Identify value,
2. Map the value stream,
3. Allow the continuous flow of activities identified as value stream,
4. Enable value pull to the customer,
5. Seek perfection.

Hence, defining and understanding customer value has become a key element in Agile projects. [8, p. 80]. It should be emphasized that rules such as “do one thing from beginning to end,” drop “batch-and-queue” processes, use the advantages of the “pull” over “push” and the requirement of action transparency for the whole team and all project stakeholders are successfully implemented in Agile projects [1].

Following the authors of Lean Thinking [7, p. 549, 9, pp. 19–20] slightly modified seven categories of waste occurring in IT projects were introduced to Lean Software Development concept. Given the problematic issues noted above, two of those categories should be highlighted: the second one, “over-production of useless goods”, called by the authors of Lean Software Development the “extra properties”,

²Agile principles were modeled after the pillars of the Toyota Production System relating to processes, people, tools and technologies: 1. Individuals and interactions over process and tools. 2. Working code over comprehensive documentation. 3. Customer collaboration over contract negotiation. 4. Responding to change over following a plan [6].

and the eighth one, “design of goods and services that do not meet customer needs,” which was omitted [1, p. 74] because it includes this type of loss that is associated with unnecessary software features that—as a result of changes in business or poorly recognized needs—have become unnecessary [10]. Given the high cost of creating such outdated or extra features, it can be said that both software functionalities that do not match current needs and redundant software functionalities lead to the production of overly expensive software, or losses stemming from creating unnecessary applications.

6 Product Backlog and Business Value as Primary Priority Factors

The traditional set of functional and non-functional requirements as the basis for software code in the Agile approach is called a product backlog. Its elements should correspond to the actual work to be done, that is, above all, the current needs of the client, but also other elements, necessary from a technical point of view [2, p. 235, 11, p. 79]. Depending on the complexity of the application, backlogs can vary in sizes, but it is recommended that they do not exceed more than 100–150 items [2, p. 331].

Product backlog is subject to a two-way action: developing and updating [11, p. 79]. The whole team, and if necessary also other stakeholders, are involved in its development. The backlog is created on the basis of a product roadmap or vision. Developing means adding new elements, and raising their level of detail from a predefined set to explicit quantitative indicators, called acceptance criteria. Each new item must be analyzed according to the evaluation criteria and then positioned or prioritized. Items arising from the technical point of view of the developer are also added to the backlog. Items added by the developer as facilities or technical innovations, unreported by the user yet raising the cost of development, may also appear on the backlog. Updating requires item organization based on the removal of items already implemented or those who completely lost value, as well as making changes, most often resulting from customer’s needs.

In the context of the problem under consideration, an important activity associated with each backlog, regardless of its level of detail, is to organize all the items by priority. Priority refers to both high level positions, corresponding to epics, essays, or themes,³ or medium or low level positions, corresponding to medium or small user stories [11, p. 80]. Prioritizing is a rather complicated decision-making process, during which criteria such as value, cost, uncertainty and risk and technological dependencies [8, pp. 80–86] undergo examination. Quantitative or semi-quantitative methods, e.g. Wiegers method [12] are possible to apply. The

³Terms vary depending on team customs or adopted method, e.g. Scrum.

responsibility for determining the priority lies on product owner as a representative of the business.

Priority determines the order of providing the customer with operating software gains. In the case of many high-priority backlog items constitutes a threat to supplying what the customer needs the most at a given time⁴ [11, p. 87]. Consequently, it is necessary to provide an additional factor, enabling the elimination of such ambiguities and support of informed decisions about priority. It is believed that the elimination of unnecessary backlog items should include features without which the application will work properly, and contain desirable features in sufficient quantities, as well as those expected, the lack of which causes the inevitable dissatisfaction of both customer and user.

It is assumed that in the case of technical restrictions, unknown to the business side of a project, the developer can participate in the process of prioritization. In such a case, a two-way exchange of information is necessary, leading to the understanding of the decision-making problem, which should not be solved alone neither by the customer, nor by the developer. Only after the values of these restrictions are known and considered by the customer can he make an informed decision about how to qualify the features or how to eliminate them.

Apparently, in difficult decision-making cases, leaving the decision to the customer alone results in 85 % of backlog items receiving high priority, 10 % medium, and only 5 % low [16], which certainly does not facilitate the management of requirements. For rapid software development, Steve McConnell proposes to filter these requirements by eliminating those non-essential [13].

7 Applying the Kano Model to Classification and Elimination of Product Backlog Items

In order to solve the problem posed above, the following assumptions were adopted:

- simplicity of the method,
- possibility of good communication between stakeholders
- language understandable to all stakeholders,
- identification of application features unnecessary to the studied segment of users.

The Kano model allows for the classification of product features into three specified categories related to varied customer satisfaction. As a result, it is possible to specify which of the described product features immediately affect the increase or decrease in customer satisfaction. The upper-left quadrant of the model (Fig. 1) presents the fastest growth of satisfaction—these are so-called attractions. In

⁴Use case applied in UML language may be another artifact.

contrast, the slowest increase in satisfaction, further limited to a maximum average level, is shown in the lower-right quadrant. The middle part of the diagram shows a linear increase in satisfaction, which means that the more such features, the greater the chance for satisfaction growth. The model shows that various strategies in the pursuit of customer satisfaction are possible, but it should be a conscious decision of the client.

Learning about customer and user preferences in relation to meeting their needs requires an interview. Two questions for each of the tested features are asked, and then, based on the analysis of the distribution of their assessments, a category is assigned to each feature. Research questionnaire for this study was constructed with the principle of assessment of satisfaction level from two opposite sides: the presence and absence of a quality [2, pp. 79–120]. In each question, the respondent is to choose one out of five options on an assessment scale:

1. I like it
2. I expect it
3. I am neutral
4. I can live with it
5. I dislike it

The evaluation of these answers based on the analysis of both sides, giving 25 combinations, allows to assign the studied features into three model categories:

- must have
- linear—linearly correlated with user satisfaction, the more/less the better/worse
- exciter—providing great satisfaction.

The other possible combinations of response pairs are further assigned into the following three categories: reverse, questionable and indifferent.⁵

8 New Template for User Story Narrative and Evaluation

The above method is suggested for categorization of product backlog items to reduce the burden of establishing their priority, and, above all, to eliminate unnecessary features which will be considered waste in the development process. Product backlog items are written on slips of paper using a specific template and natural language, which ensures that the text of user story narrative is clear to all project stakeholders. In accordance with the Lean principle of transparency, visualization of work is achieved with the use of a system known as “card walls” [15].

⁵Description of matrix interpretation rules for cross-reference assessment of functional and dysfunctional questions used to categorize the responses and to calculate the indicators of their distribution can be found in [14, pp. 79–120, 2, pp. 114–117].

Table 1 Structure of the new user story template, combined with the user assessment question.
Source Author

First part of user story phrase	First part of Kano method question	Second part of user story phrase	Second part of Kano method question	Response scale	Mark the answer with an X
<i>As a <type of user></i>	<i>If I can or If I can not</i>	<i>I want <some goal> so that <some reason></i>	<i>How do I feel?</i>	<i>I like it</i>	
				<i>I expect it</i>	
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	

In order to apply the Kano method to product backlog, it is proposed to extend the user story narrative template presented below [2, p. 239]:

As a <type of user > , I want <some goal > so that <some reason >

(1)

where:

- *type of user*—the person (role), which expects the product, profit,
- *some goal*—awaited feature, property of the product,
- *some reason*—profit or value of the feature

with the two questions posed to the respondent, as well as the scale of possible answers to choose from. Due to the discrepancies in personal forms used in the user story template and Kano questions, it is proposed to standardize them and adopt the forms characteristic of user story, substituting “you” with “I” in Kano method questions. In case of functional questions the full phrase will be “if I can,” and in dysfunctional—“if I cannot”; the phrasing of Kano questions will also gain a second part: “how do I feel?”. The extended template for both functional and dysfunctional questions is presented in Table 1. The structure of these questions is created in natural language and does not contain technical terms incomprehensible to the customer.

As a representative of the business, the owner of the product who manages product backlog should ensure that an opinion poll is conducted among a substantial group of users in the course of writing the content of user stories. After completing the records in line with the specified template (Table 1), it is possible to analyze the distribution of the responses. The outcome is a list of registry items classified into one of six Kano model categories. This allows to take a reasoned decision on the elimination of unnecessary user stories, in line with the adopted elimination strategy. An example strategy for user stories from the “linear” category may read as follows: items with an distribution index value <90 % should be eliminated, in the “questionable” category elimination should apply to all user story regardless of the distribution index value.

Table 2 Usage example of the proposed template. *Source* Author

First part of user story phrase	First part of Kano method question	Second part of user story phrase	Second part of Kano method question	Response scale	Mark the answer with an X
<i>As a lecturer (1)</i>	<i>If I can</i>	<i>I want to change the allocated course so that the schedule is correct</i>	<i>How do I feel?</i>	<i>I like it</i>	X
				<i>I expect it</i>	
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	
<i>As a lecturer (1)</i>	<i>If I can not</i>	<i>I want to change the allocated course so that the schedule is correct</i>	<i>How do I feel?</i>	<i>I like it</i>	
				<i>I expect it</i>	
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	X
<i>As a lecturer (2)</i>	<i>If I can</i>	<i>I want to watch course allocations on a graphic table so that the search is faster</i>	<i>How do I feel?</i>	<i>I like it</i>	
				<i>I expect it</i>	X
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	
<i>As a lecturer (2)</i>	<i>If I can not</i>	<i>I want to watch course allocations on a graphic table so that the search is faster</i>	<i>How do I feel?</i>	<i>I like it</i>	
				<i>I expect it</i>	
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	X
<i>As a lecturer (3)</i>	<i>If I can</i>	<i>I want to accept all course allocation so that I am not surprised with it</i>	<i>How do I feel?</i>	<i>I like it</i>	X
				<i>I expect it</i>	
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	
<i>As a lecturer (3)</i>	<i>If I can not</i>	<i>I want to accept all course allocation so that I am not surprised with it</i>	<i>How do I feel?</i>	<i>I like it</i>	X
				<i>I expect it</i>	
				<i>I am neutral</i>	
				<i>I can live with it</i>	
				<i>I dislike it</i>	

The topic “Effective course planning” serves as an example that illustrates the concept of creating an enlarged structure of the user story template (Table 2).

Analysis of the responses conducted in line with the rules of the cross-reference matrix in Kano model indicates that the user story no. 1 belongs to the linear category, user story no. 2 to must-have category, while no. 3 goes in the category of questionable assessment, suggesting a mistake in assessment process the presence or lack of interest in such a possibility.

Therefore, a simple analysis of responses from representative users will allow for a reliable categorization of their requirements, while the product owner responsible for prioritizing product backlog items can make a correct decision that follows the strategy arising from the business needs. This solution will allow a release of products in different versions, depending on the actual needs of users.

9 Conclusion

Reinforcing the principles of Agile approach through Lean and the establishment of Lean Software Development has become the source of the presented concept, focused on early elimination of unnecessary software features. Waste of time and resources on redundant or obsolete features of IT solutions is not infrequent, but quite common—as shown in references and in author’s pilot study. The tools existing in the studied approach have been used to design an attempt at solving this problem. Simplicity, both of the Kano model and of the user story template structure, customer-friendly environment of the Agile approach which facilitates the required communications between customer and developer, the “card walls” method of work visualization and, above all, the guiding principle of waste elimination should all be considered favorable circumstances in the application of this project.

It is emphasized that the two aspects of product backlog management—the development and update of backlog items with simultaneous value assessment—combined in this concept are subject to the principles of Agile and Lean approaches. Their aim is to reduce the amount of work in progress and the opportunity to collect records, which consequently leads to a waste of time on unnecessary product features and failure to provide the software valued by customer on time.

However, the proposed extension in the structure of user story recording with the assessment question on individual user experience requires further testing, in order to verify its effectiveness and possible nuisances in practical application. It is believed that teams accustomed to the currently used user story template will not be surprised by this extension, and when the extended recording structure will become generated automatically by Agile project management tools, it will become a practical way to support the elimination of waste. Employing computers to aid this solution will open up to a much easier use of the more complex structure, at the same time providing documentation of its practicality, which will be the target of further studies of the author, and will allow to gain confidence in the solution.

References

1. Poppendieck, M.T.: Implementing Lean Software Development. From Concept to Cash. Addison-Wesley, New York (2007)
2. Cohn, M.: Succeeding with Agile. Software Development Using Scrum. Addison-Wesley, Michigan (2010)
3. PN-EN ISO 9000:2001: Systemy zarządzania jakością. Podstawy i terminologia. Polski. Komitet Normalizacyjny, Warsaw (2001)
4. <http://www.uio.no/studier/emner/matnat/ifi/INF5181/h11/undervisningsmateriale/reading-materials/Lecture-06/04ZubrowISO25000SWQualityMeasurement.pdf>. Accessed 20 July 2016
5. https://en.wikipedia.org/wiki/Shareholder_value#Shareholder_Value_Maximization, Accessed 20 July 2016
6. The manifesto for Agile software development. www.agileAlliance.org. Accessed 20 July 2016
7. Womack, J.P., Jones, D.T.: Lean Thinking—Szczupłe Myślenie. Eliminowanie marnotrawstwa i tworzenie wartości w przedsiębiorstwie. ProdPress, Wrocław (2008)
8. Cohn, M.: Agile Estimating and Planning. Addison-Wesley, Michigan (2006)
9. Womack, J.P., Jones, D.T., Roos, D.: The Machine That Changed the World. Rawson Associates (1990)
10. Hołodnik-Janczura, G.: Kategorie marnotrawstwa w projektach IT. *Ekonomika i organizacja przedsiębiorstw*, No. 9, pp. 31–40 (2013)
11. Pichler R.: Zarządzanie projektami ze Scrumem. Twórz produkty, które pokochają klienci, Helion (2014)
12. Wieggers, K.E.: First Thing First: Prioritizing Requirements. *Software Development*, September 1999
13. McConnell, S.: Rapid Development. Microsoft Press (1996)
14. Cohen, L.: Quality Function Deployment. Addison-Wesley Pub. Comp, USA (1995)
15. Anderson, D.J.: Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press, Washington (2010)
16. Wieggers, K.E.: First things first: prioritizing requirements. <http://www.processimpact.com/articles/prioritizing.html>. Accessed 20 July 2016